

Contents

Neural Network Quine: a reimplementa- tion, and making it work	1
Summary	1
1. The setup	2
2. Why SSE alone is not enough	2
Part I: reproducing the paper	2
3. Reproduction status	2
4. Gradient training (paper E1, E2)	3
5. Regeneration (paper E5, E6)	4
5.1 Regeneration with one Adamax epoch per generation	4
5.2 Regeneration without training	4
6. Hill-climbing (paper E3, E4)	4
7. The MNIST version (paper E7, E8)	4
8. The paper’s derived metrics	4
9. Initialization: the paper’s text and its numbers disagree	5
Part II: making the quine copy itself	5
10. The paper’s own setup learns, if trained longer	5
11. What the starting point and architecture change	5
12. The learning rate was the main obstacle	6
13. Changing what the gradient does: full gradient, and $1 - R^2$ as the loss	6
14. Solving the quine equations directly: Newton’s method	7
15. The best network	8
16. Reading off versus computing: does a guess depend on its own weight?	9
17. Remaining uncertainty and open questions	10
Reproduction	10

Neural Network Quine: a reimplementa- tion, and making it work

Paper: Oscar Chang and Hod Lipson, “Neural Network Quine”, ALIFE 2018, arXiv:1803.05859v4. Every run mentioned here is listed, with all its settings and its final numbers, in the registry in EXPERIMENTS.md; the trained networks are saved in results/weights/. Unstated details of the paper and how each was resolved are in PLAN.md; the working log is in NOTES.md.

Summary

A neural network quine is given the index of one of its own weights and must output that weight. I reimplemented the paper from its text (Part I) and then worked on making the network actually reproduce its weights (Part II).

Success is measured here by $R^2 = 1 - \text{SSE} / \sum(\theta - \text{mean } \theta)^2$, where SSE is the sum of squared errors between the network’s guesses and its weights θ . $R^2 = 1$ is a perfect copy; $R^2 = 0$ is no better than guessing the mean weight. The paper reports only SSE, which a network can lower without copying anything, by shrinking its weights and guessing zero.

Part I. With an initialization chosen to match the paper’s starting loss, most of the paper’s numbers reproduce: the starting SSE (86.7 against 90.16), the ranking of optimizers, the SSE reached by regeneration (0.43 to 0.88 against 0.86), and the MNIST accuracy (89.7% against 90.41%). But no network trained with the paper’s methods for the paper’s training lengths reaches an R^2 above 0.10, and most end between -0.05 and 0.02 : they guess roughly zero, and the low SSE comes from small weights. Two claims did not reproduce: the accuracy cost of self-replication (a

classifier alone reaches the same 89.7%, not 96.33%), and the collapse of pure regeneration to zero (it diverges instead, for this initialization).

Part II. The quine does work, given the right setup and enough optimization. In order of importance: training far longer than the paper’s 100 epochs; He initialization of the weights with a unit-scale address table; a lower learning rate; and finally damped Newton (Levenberg–Marquardt), which solves the quine equations directly. Two changes to the loss matter from the start of training: taking the full gradient (the targets are the live weights) and dividing the loss by the weights’ spread, so that it equals $1 - R^2$ (section 13). With all of these, the best two-layer network (the paper’s architecture) reaches $R^2 = 0.9975$, with normal-sized weights (RMS 0.90); all three two-layer seeds reach 0.994 before damped Newton (section 13). Long damped Newton runs appeared to stall, but the stall came from the damping schedule, not from the problem: restarting the method resumes progress (section 14).

1. The setup

The network is $f(c) = w^T \sigma(W_2 \sigma(W_1 \sigma(p_c)))$ (two hidden layers, the paper’s) or $f(c) = w^T \sigma(W_1 \sigma(p_c))$ (one hidden layer). Here c indexes a weight, p_c is row c of a fixed random table P (100 numbers per weight, never trained, not part of the weights), σ is the SELU activation, and the trained weights θ are W_1 , W_2 (100×100 each) and w (100), so $N = 20,100$ (two layers) or 10,100 (one layer). The paper writes the table lookup as a one-hot vector times a fixed random projection; the two are the same computation. A learnable table would have far more entries than the network it describes.

Training, as in the paper: each epoch freezes a copy of the weights as targets and visits every weight once in random minibatches of 10. The paper also uses hill-climbing (random perturbations kept if they lower the minibatch SSE) and regeneration (replace every weight by the network’s guess of it). The MNIST version feeds an image alongside the address and adds a classifier head. EXPERIMENTS.md defines every variant precisely.

2. Why SSE alone is not enough

A network that outputs 0 for every weight has $SSE = \sum \theta^2$, the sum of its squared weights. So any change that shrinks the weights lowers SSE, whether or not the network learns anything about itself, and the all-zero network has $SSE = 0$. The paper notices the all-zero solution but reports no quantity that separates shrinking from copying. R^2 does: shrinking the weights does not change it, and guessing zero gives $R^2 \approx 0$ (exactly 0 if the mean weight is 0).

Part I: reproducing the paper

3. Reproduction status

Mean (sd) over seeds 0 to 2 unless noted; “~” marks values read off the paper’s figures. R^2 is at the end of each run.

quantity	paper	this reimplementatation	R^2 (mine)
starting SSE	90.16	86.7 (5.8), 20 seeds	negative (−0.25 for seed 0)
SGD, 30 epochs	plateau ~66	64.8 (0.1)	0.02
SGD with momentum, 30 epochs	~64.5	54.3 (1.1)	0.04 to 0.07
Adam, 30 epochs	~56.5	68.0 (0.8)	≈ 0
Adagrad	rises over time	best 64.2 at epoch 4, then rises to 74.9	0.09
RMSprop	explodes	rises to 671 (165)	negative

quantity	paper	this reimplementation	R ² (mine)
Adamax, 100 epochs	best 32.10, at the end	best 41.8 at epoch ~25, then rises to 58.8	0.01 to 0.02
hill-climbing, 10,000 epochs	90 → ~64	71.3 at epoch 2,400, then lost (section 6)	≈ 0
hill-climbing from the SGD solution	improves it significantly	no improvement (65.7 → 67.7)	0.01
regeneration (1 Adamax epoch per generation, 10 generations)	best 0.86	best 0.88, 0.61, 0.43	−0.05 to −0.01
regeneration without training	collapses to all-zero weights	diverges, 3 of 3 seeds	
MNIST version, starting combined loss	1072.05	1063.9 (53.0)	
MNIST version, SSE over training	rises ~80 → ~260	falls 138 → 63	≈ 0
MNIST version, test accuracy	90.41%	89.69% (0.81)	
same network, classification only	96.33%	89.65% (0.71)	

4. Gradient training (paper E1, E2)

Observation. The optimizers reproduce the shape of the paper’s Fig. 4 (figure: figures/e1_optimizers.png). All but RMSprop drop from ~85 to between 62 and 67 in the first epoch; SGD then plateaus near 65, Adagrad turns upward after epoch 4, RMSprop diverges, and Adamax goes lowest.

Observation. After the first epoch, SSE stays within 10% of $\Sigma\theta^2$ (between 0.90 and 1.05 times it, every optimizer but RMSprop, all epochs and seeds). The first-epoch drop is the network learning to output almost nothing: for Adamax, seed 0, the prediction RMS falls from 0.030 to 0.0067 while the weight RMS stays at 0.056. From then on SSE and $\Sigma\theta^2$ fall together:

Adamax, seed 0, epoch	0	1	5	10	24	50	100
SSE	83.8	62.5	55.0	48.4	41.9	42.8	53.7
$\Sigma\theta^2$ (SSE of guessing zero)	66.8	62.1	55.2	48.7	42.4	43.2	54.8
prediction RMS	0.030	0.0067	0.0047	0.0053	0.0056	0.0064	0.0069

Interpretation. Within 100 epochs, gradient training finds the guess-zero solution in the first epoch and then lowers SSE by shrinking the weights. (Part II shows that the same setup does start to learn after epoch ~50, which is why SSE rises again toward epoch 100.)

My Adamax minimum (41.8) is higher than the paper’s 32.10. With SELU also on the output layer, an unstated detail, Adamax reaches 32.3 by epoch 30, again with $R^2 \approx 0$.

5. Regeneration (paper E5, E6)

5.1 Regeneration with one Adamax epoch per generation

Observation. SSE measured after each regeneration is 0.43 to 1.1 across three seeds, the paper’s order of magnitude (0.86). The weight RMS there is 0.0046 to 0.0074, a tenth of its starting value, and R^2 is -0.05 to -0.01 (figure: figures/e5_regeneration.png). The paper says of this solution that “the order of magnitude of the weights are in line with what we would observe in a normal neural network”; in my runs the weights are about the size of the prediction error.

Mechanism (diag_regen_cycle.py, output in results/diag_regen_cycle.txt). Call the weights just after a regeneration θ_R . During the next Adamax epoch, the network learns to output θ_R (to within 0.2% to 2.3% of $\Sigma \theta_R^2$), while its own weights move to $\theta_R + \Delta$ with $|\Delta|^2 \approx |\theta_R|^2$. Regeneration then writes θ_R back, and the network with weights θ_R outputs almost nothing (prediction RMS $7e-5$). So regeneration alternates between two networks: one prints the other, and the other prints nothing. Neither prints itself.

5.2 Regeneration without training

Observation. All three seeds diverge: the weight RMS falls for one or two generations and then grows without bound until the loss overflows.

Mechanism (diag_regen_scale.py). Repeating pure regeneration with the starting weights multiplied by α : $\alpha \leq 0.5$ collapses to exactly zero within four to six generations (all seeds); $\alpha \geq 1$ diverges (all seeds); $\alpha = 0.75$ goes either way. With every weight scaled by ϵ , the output scales roughly as ϵ^3 (the table P does not scale), so all-zero weights attract strongly, but only within a basin. The paper’s collapse and my divergence are opposite sides of the same boundary.

6. Hill-climbing (paper E3, E4)

Observation. In a 50-epoch sweep of the perturbation size σ , the acceptance rate is 49 to 52% at every σ . Larger σ makes SSE and $\Sigma \theta^2$ grow together ($\sigma = 1e-3$: $\Sigma \theta^2$ from 67 to 1043). Each step moves all 20,100 weights but checks only 10 predictions, so accepted steps are close to a random walk, which inflates the weights.

Observation. Hill-climbing from the SGD and Adamax solutions (1,000 epochs, $\sigma = 1e-5$ and $3e-5$) improved neither; the paper reports that it improves the SGD solution. All end with $R^2 \approx 0$ (-0.002 to 0.017).

Gap. The two 10,000-epoch runs of E3 were lost to a container restart at epoch 2,400 (SSE 71.3 at $\sigma = 1e-5$ and 110.4 at $\sigma = 3e-5$; the paper’s curve is ~ 69.5 there, ~ 64 at the end; $R^2 \approx 0$ throughout). They were not rerun.

7. The MNIST version (paper E7, E8)

Observation. The starting losses match (1063.9 against 1072.05) and so does the accuracy after 30 epochs (89.69% against 90.41%). But SSE falls ($138 \rightarrow 63$, $R^2 \approx 0$) where the paper’s rises, and the same network trained only to classify reaches the same accuracy (89.65%), not the paper’s 96.33%.

Interpretation. The self-replication term is satisfied by guessing zero, which costs the classifier nothing, so there is no trade-off to observe. I cannot explain the paper’s 96.33% baseline; it may have been trained differently (the paper does not say).

8. The paper’s derived metrics

Both formulas are inferred from the paper’s printed numbers, which match them to every digit. The “average weight prediction margin”, described as the mean absolute error, equals the RMS

error $\sqrt{(\text{SSE}/N)}$: $90.16 \rightarrow 0.067$, $32.10 \rightarrow 0.040$, $0.86 \rightarrow 0.0065$. The “self-replicating quotient”, described as a log likelihood ratio against random guessing, equals $\ln(N/\text{SSE})$: 6.44 and 10.06. Taken literally, the chance of a random point landing within $\sqrt{\text{SSE}}$ of the weights in 20,100 dimensions would give quotients of tens of thousands. Both metrics are functions of SSE alone, so neither can tell copying from shrinking.

9. Initialization: the paper’s text and its numbers disagree

The paper says He initialization. An untrained network’s SSE is about $\Sigma\theta^2 + \Sigma f^2$, and He weights alone give $\Sigma\theta^2 \approx 402$, so the paper’s starting SSE of 90.16 is impossible with them (I measured $\sim 95,000$ with a unit-scale table). PyTorch’s default initialization (uniform on ± 0.1) with the table at the same scale gives 86.7 (5.8). The same choice reproduces the MNIST version’s starting losses. My inference, with moderate confidence, is that the authors’ code used PyTorch’s defaults. Part I uses them throughout.

Part II: making the quine copy itself

All of Part II uses the plain quine (no MNIST). R^2 values are per seed (0 / 1 / 2) unless stated.

10. The paper’s own setup learns, if trained longer

Observation. With the paper’s setup (two layers, default initialization), training for 1,000 epochs instead of 100 changes the outcome. The weights shrink until about epoch 50, where SSE reaches its minimum (~ 42 , close to the paper’s best), and then grow while the network increasingly copies itself:

epoch (seed 0)	25	50	100	200	400	600	1,000
R^2	0.00	0.00	0.01	0.07	0.20	0.31	0.44
SSE	42	43	54	137	439	784	1,291
weight RMS	0.046	0.046	0.052	0.085	0.165	0.238	0.341

Interpretation. SSE rises as the copy improves, because SSE grows with the size of the weights. Selecting the run with the lowest SSE, as the paper does, picks the moment just before learning starts.

11. What the starting point and architecture change

Observation. Weight initialization and the scale of P, crossed, with one and two hidden layers (Adamax at the default rate, 1,000 epochs, 3 seeds, R^2 averaged over the last 100 epochs):

weights, P scale	two layers	one layer
default, 0.058 (the paper’s)	0.39 to 0.43	0.45 to 0.56
default, 1	≈ 0 ; weights collapse to RMS 0.02	≈ 0 ; weights collapse to RMS 0.01
He, 0.058	0.55 to 0.57	0.79 to 0.80
He, 1	0.84 to 0.86	0.87 to 0.89

- He initialization with a unit-scale table is best. I had predicted the opposite ranking of the two factors, from the size of the activations at initialization; the prediction was wrong.
- Small weights with a large table is the one combination that never leaves the guess-zero state. I have not investigated why.

- One hidden layer copies itself as well as two or better, despite (or because of) having half as many weights to reproduce. Untested explanation: more capacity per weight that must be copied.
- Applying SELU to the looked-up row of P, or not, makes no measurable difference (R^2 0.850 against 0.848, He initialization, two layers).

12. The learning rate was the main obstacle

Diagnostic (diag_gap.py). Is the remaining error a failure to fit the frozen targets, or drift of the weights within an epoch? From the networks after 1,000 epochs: drift is negligible (under 0.1% of the weights' variance per epoch), but one extra epoch at a lower learning rate jumps R^2 from 0.87 to 0.93 (one layer) and from 0.87 to 0.91 (two layers). Adamax moves each weight by up to the learning rate at every step, so at the default $2e-3$ the weights jitter around a better solution.

Observation. Continuing at lower rates (He initialization, P scale 1):

stage, each continuing the previous	one layer	two layers
Adamax lr $2e-3$, 1,000 epochs	0.870 / 0.866 / 0.900	0.874 / 0.837 / 0.834
+ 300 epochs at lr $2e-4$	0.947 / 0.959 / 0.966	0.945 / 0.933 / 0.947
+ 300 epochs at lr $2e-5$	0.955 / 0.964 / 0.970	0.955 / 0.943 / 0.955

13. Changing what the gradient does: full gradient, and $1 - R^2$ as the loss

Two changes to the loss, tested from the lr $2e-5$ networks with 300 more epochs at lr $2e-5$:

- Full gradient: the targets are the live weights, so each step also moves each weight toward its own prediction (the paper freezes the targets).
- $1 - R^2$ as the loss: divide the minibatch SSE by the weights' current spread, so shrinking the weights cannot lower the loss.

variant	one layer	two layers
frozen targets, SSE (control)	0.958 / 0.966 / 0.971	0.961 / 0.951 / 0.961
full gradient, SSE	0.960 / 0.967 / 0.973	0.962 / 0.952 / 0.962
full gradient, $1 - R^2$	0.960 / 0.967 / 0.973	0.962 / 0.952 / 0.962
frozen targets, $1 - R^2$	not run	0.961 / 0.951 / 0.961

Observation. The full gradient helps by a small, consistent amount (about 0.0015 on every seed of both architectures). Using $1 - R^2$ as the loss changes nothing at this stage.

Interpretation. $1 - R^2$ differs from SSE only by the weights' spread, and the weights were not shrinking at this stage (RMS steady at ~ 0.40), so the divisor is nearly constant and Adamax ignores a constant rescaling of the loss. With frozen targets the divisor is exactly constant within an epoch, so no difference was expected, and none appears (two layers: identical to four decimal places on all three seeds).

From random initialization the picture changes. Same four variants, two layers, He initialization, P scale 1, Adamax at the default lr $2e-3$, 1,000 epochs (final R^2 , seeds 0 / 1 / 2; weight RMS at the end):

variant	final R^2	best R^2 during the run	weight RMS
frozen targets, SSE (baseline)	0.874 / 0.837 / 0.834	0.876 / 0.855 / 0.877	0.39

variant	final R^2	best R^2 during the run	weight RMS
frozen targets, $1 - R^2$	0.875 / 0.852 / 0.860	0.875 / 0.854 / 0.873	0.39 to 0.40
full gradient, SSE	0.847 / 0.848 / 0.833	0.868 / 0.867 / 0.865	0.19 to 0.20
full gradient, $1 - R^2$	0.940 / 0.942 / 0.939	0.950 / 0.954 / 0.956	0.90 to 1.16

Observation. The full gradient with $1 - R^2$ as the loss beats the baseline by 0.07 to 0.10 on every seed, with the same budget. Either change alone does not help: $1 - R^2$ with frozen targets matches the baseline, and the full gradient with SSE is worse, because it shrinks the weights (RMS 0.19 against 0.39).

Interpretation. The full gradient pulls each weight toward its own prediction; early in training the predictions are small, so with SSE this pull shrinks the weights, which SSE rewards. Dividing by the weights' spread removes that reward, and the same pull then helps: the weights grow (RMS ~ 1) and the copy improves. The two changes matter in combination and only early in training, which is why the continuation runs above showed no effect.

Continuing these from-scratch networks (full gradient, $1 - R^2$) with lower learning rates, 300 epochs each:

stage	R^2 (seeds 0 / 1 / 2)	weight RMS
1,000 epochs at lr 2e-3 (the table above)	0.940 / 0.942 / 0.939	0.90 / 1.17 / 1.08
+ 300 epochs at lr 2e-4	0.9926 / 0.9927 / 0.9926	0.90 / 1.17 / 1.08
+ 300 epochs at lr 2e-5	0.9941 / 0.9941 / 0.9945	0.91 / 1.17 / 1.08

Observation. The three seeds end within 0.0004 of each other, at 0.994, with the weights at their full size. The old pipeline (frozen targets and SSE, then the same learning-rate steps, then 20 damped Newton iterations) reached 0.971 on seed 0 (section 14). Adamax with the right loss now beats it, before any damped Newton.

14. Solving the quine equations directly: Newton's method

The quine condition $f(c) = \theta_c$ for every c is N equations in N unknowns. With residual $r = f(C) - \theta$ and J the Jacobian of the outputs with respect to the weights, Newton's method solves $(J - I)\Delta = -r$.

Observation, pure Newton (diag_newton.py). It fails at the first step. The Jacobian is correct (checked against finite differences), but $J - I$ is badly conditioned (condition number 4.8 million), the Newton step is 6.75 times the size of the weights, and the linear model holds only for steps about 10^{-6} of that size.

Observation, damped Newton (Levenberg-Marquardt), which solves $(A^T A + \mu I)\Delta = -A^T r$ with $A = J - I$ and an adaptive damping μ :

run	start	after 20 iterations
one layer (from the full-gradient networks)	0.960 / 0.967 / 0.973	0.983 / 0.985 / 0.987
two layers, seed 0 (from the lr 2e-5 network)	0.955	0.971, still rising

One iteration takes about 20 seconds for one layer and 2 to 3 minutes for two layers, mainly the Jacobian and an $N \times N$ linear solve.

Observation, longer and normalized. Continuing the one-layer seed-0 network for 80 more iterations: $0.9832 \rightarrow 0.9852$ minimizing SSE, and $0.9832 \rightarrow 0.9853$ minimizing $1 - R^2$ (which also keeps the weight RMS fixed at 0.391 instead of shrinking it to 0.390). Both stall, with the damping rising and the steps shrinking to 10^{-5} of the weights.

Correction: this is not a local optimum. Restarting damped Newton on $1 - R^2$ from the stalled network gives $0.9853 \rightarrow 0.9884$ in 10 iterations (logs/diag_absorb.died-at-restart.log, control run). The stall came from the damping schedule: μ is multiplied by 4 after a rejected step and divided by 3 after an accepted one, so over a long run it ratchets up until the steps are too small to matter; a restart resets it. Also, changing the weights by a random 1% and then repairing them reached 0.9901 in 9 iterations, better than the unchanged network over the same iterations (that network was lost in a container restart; the rerun gave 0.9904, and the two-layer version is in section 16).

Interpretation. Damped Newton is far more efficient than Adamax here (the gain from 0.96 to 0.98 took 20 iterations). Changing the objective from SSE to $1 - R^2$ makes no difference for one layer or two (two layers, seed 0, 20 iterations: 0.9713 on SSE, 0.9714 on $1 - R^2$).

Observation, two damping rules (two layers, seed 0, from the 0.9941 network of section 13, damped Newton on $1 - R^2$, about 90 to 105 seconds per iteration):

run	after 5	after 10	after 20 iterations
old rule ($\mu \times 4$ after a rejected step, $\div 3$ after an accepted one)	0.99553	0.99653	
old rule, restarted after 10 iterations (μ reset)		0.99653	0.99753
Nielsen's rule (Nielsen 1999; implemented in newton.lm)	0.99508	0.99577	0.99659

The old rule's first 10 iterations were run twice (the control of the change-and-repair experiment in section 16, and the first half of the restart run) and gave the same result, as they must: the method is deterministic.

Observation. Restarting the old rule every 10 iterations beat Nielsen's rule by 0.0009 at 20 iterations. Under Nielsen's rule the damping stayed near 5×10^{-4} and the steps near 0.3% of the weights' size; the old rule, just after a reset, took steps of 0.5 to 1.3%.

Interpretation (one seed, one start). On this problem, larger steps pay, and Nielsen's rule is too cautious. The best schedule tried so far is the old rule with a reset every 10 iterations. The ceiling, if there is one, is not yet known; R^2 was still rising by about 0.00004 per iteration at the end.

15. The best network

The best network (two layers, seed 0, R^2 0.99753; results/weights/lmnormReset_L2fn_seed0_b.pt) has normal-sized weights: RMS 0.90 (0.14 at initialization), median magnitude 0.46, largest 8.4. The error RMS is 0.045, 5% of the weight RMS. The error is spread evenly: weights with errors more than three times the typical error carry 4% of the total, close to what bell-curve noise would give.

By block:

block	weights	R ² within the block	share of the total SSE
W ₁	10,000	0.9944	52%
W ₂	10,000	0.9985	48%
w (output weights)	100	−1.20	0.5%

The output weights are small (standard deviation 0.030, largest 0.043), and they are guessed with the same absolute error as all the others (about 0.045). Their errors are larger than their spread, so R² within that block is negative. They carry too little of the total to affect the overall R².

The best one-layer network (seed 2, R² 0.987) was described in an earlier version of this section: weight RMS 0.41, error RMS 0.045, output-weight R² 0.86.

16. Reading off versus computing: does a guess depend on its own weight?

A network could match its weights in two ways. It could read them off, as a lookup would. Or it could compute values that happen to equal them. The test: nudge one weight θ_c and see whether the guess $f(c)$ for that weight moves with it. The self-sensitivity $\partial f(c)/\partial \theta_c$, the diagonal of the Jacobian J, is 1 for a lookup (diag_grounding.py).

Observation (two layers, seed 0, R² 0.9941; results/diag_grounding_L2_stage3fn_seed0.txt; one layer, seed 2, R² 0.987, for comparison):

block	mean $\partial f(c)/\partial \theta_c$, two layers	mean $\partial f(c)/\partial \theta_c$, one layer
W ₁	0.0009	0.0001
W ₂	0.0008	(no such block)
w (output weights)	4.07	0.95

The length of a row of J, the sensitivity of one guess to all weights together, has median 121 (two layers) and 38 (one layer).

Observation, output weights. For an output weight w_j , $\partial f(c_j)/\partial w_j = h_j(c_j)$ exactly: the activation of last-layer unit j at w_j 's own index. This is not a dedicated lookup unit. At output-weight indices, $h_j(c_j)$ has mean 0.95 and standard deviation 2.84 (one layer), and mean 4.07 and standard deviation 9.95 (two layers). The other units are, on average, larger in magnitude (mean $|h_k(c_j)|$ 2.24 and 7.22). The one-layer mean of 0.95 is an average of widely scattered values, not a sign of lookup.

Interpretation. In both networks the guesses for the hidden-layer weights, 99.5% of the weights, hardly depend on the weight being guessed. They are right because of how all the weights are set together. An accurate self-description here is not evidence of access to the thing described.

Change and repair (diag_absorb.py; results/diag_absorb_L2.json). Starting from the same two-layer network, change the weights by a random Δ , then run 10 iterations of damped Newton on $1 - R^2$. Compare the result with a control run from the unchanged network. Retention is the component of (final − control) along Δ , as a fraction of Δ : 1 means the change was kept, 0 that it was removed. Other drift is the rest of (final − control), in units of $|\Delta|$.

run	R ² right after the change	R ² after repair	retention	other drift
control (no change)	0.9941	0.9965		

run	R ² right after the change	R ² after repair	retention	other drift
$ \Delta = 1\% \text{ of } \theta $	-0.26	0.9965	0.085	1.30
$ \Delta = 10\% \text{ of } \theta $	-152	0.9975	0.127	1.04

The singular values of $J - I$ at the start: largest 7,505, smallest 0.00002; 4 below 0.001, 34 below 0.01, 336 below 0.1, 2,976 below 1, of 20,100. A small singular value marks a direction in which a weight change is nearly followed by the guesses. For one layer: largest 1,088; 3, 19, 178 and 1,736 below the same thresholds, of 10,100.

Observation. The two-layer network is far more fragile than the one-layer one: a random 1% change sends R^2 to -0.26 (one layer: 0.81). Repair recovers fully from both changes. It keeps 9% to 13% of the change and ends about one change-size away from the control, at an equal or better R^2 . The one-layer results were similar (retention 0.09 and 0.14).

Interpretation. The near-quinies form a broad region. A change is neither kept nor undone; the network settles at a different point of the region. The 10% change ended 0.001 above the control after the same 10 iterations, as in the one-layer case. Whether a large change helps damped Newton in general is not established: there is one random change per size and one seed.

17. Remaining uncertainty and open questions

- Part I depends on an initialization inferred from the paper's numbers, not its text. The forced details (no biases, layer sizes, minibatch of 10, frozen targets) and the matched starting losses limit the room for error, but the authors' code may differ in ways that matter.
- The paper reports no weight sizes, so I cannot confirm that its networks were guessing zero; I can only say that its numbers are what guessing zero produces in this reimplement.
- Most Part II results use three seeds. The damped Newton runs for two layers, the self-sensitivity measurement, and change-and-repair use one seed (seed 0), and change-and-repair uses one random change per size.
- Whether $R^2 = 1$ is reachable at all is open: it requires a non-zero exact solution of N equations in N unknowns, and the badly conditioned Jacobian says nothing about whether one exists.
- Untested directions (IDEAS.md): structured addresses, so related weights get related addresses; smaller networks; why small weights with a large table collapse; the MNIST version with the setup that works.

Reproduction

```
./run_all.sh                # Adamax, hill-climbing, regeneration, MNIST runs (detached)
.venv/bin/python newton.py ... # damped Newton runs; arguments in EXPERIMENTS.md
.venv/bin/python registry.py  # registry of every run (EXPERIMENTS.md)
.venv/bin/python summarize.py  # tables
.venv/bin/python plots.py      # figures
```

Seeds 0, 1 and 2; PyTorch 2.14.1 on CPU.